

FibexSign

Informe de Pruebas de Caja Blanca y Fallas Encontradas

GrupoConex — Plataforma de firma electrónica self-hosted ("Firma")

Alcance: `apps/OpenSignServer` — Lote 1 (autenticación y alta de usuarios)

Generado: 2026-09-30

Hecho por: Ciclo Fibex (GDA)

Rama `test/white-box-coverage`, commit `2f9cc4f4`. Este informe documenta la cobertura alcanzada, la metodología y las fallas de seguridad y de lógica detectadas por las pruebas. Ninguna falla fue corregida en esta rama: todas quedan registradas como defectos conocidos para su corrección en una rama separada.

Índice

1. Resumen ejecutivo
2. Alcance y criterio de aceptación
3. Metodología y herramientas
4. Línea base de cobertura
5. Resultados del Lote 1
6. Fallas de seguridad encontradas (SEC)
7. Cadena de ataque: toma de cuenta por email
8. Hallazgos adicionales de seguridad
9. Defectos de manejo de errores (DEF)
10. Código muerto e inalcanzable
11. Revisión de calidad de las pruebas
12. Recomendaciones y orden de corrección
13. Plan de los lotes siguientes

1. Resumen ejecutivo

Se inició un plan de pruebas de caja blanca sobre todo el sistema FibexSign, empezando por el servidor (`apps/OpenSignServer`). El primer lote cubre las 14 funciones cloud de **autenticación y alta de usuarios**.

Indicador	Valor
Archivos cubiertos en el Lote 1	14
Cobertura del Lote 1 (líneas / funciones / ramas)	100 % / 100 % / 100 % en cada archivo
Specs en la suite del servidor	43 → 273
Fallos	0 (8 corridas consecutivas en verde)
Defectos conocidos registrados	12 (7 de seguridad SEC, 5 de manejo de errores DEF), cubiertos por 14 specs
Código de producción modificado	Ninguno

Hallazgos más graves:

- **SEC-05 (CRÍTICO)** — `adduser` sobrescribe la contraseña de cualquier cuenta existente que coincida por email, incluso de otro tenant. Combinado con el registro anónimo como Admin, permite **tomar el control de cualquier cuenta conociendo solo su email**.
- **SEC-03 (ALTO por cadena)** — `verifyemail` no tiene expiración, bloqueo por intentos ni rate limit. Funciona como oráculo para probar por fuerza bruta el OTP de 4 dígitos que luego abre sesión con `AuthLoginAsMail` : **toma de cuenta de cualquier usuario por email**.

Se recomienda abrir una rama `fix/auth-security` desde `staging` y corregir SEC-05 y la cadena de SEC-03 antes de cualquier despliegue a producción.

2. Alcance y criterio de aceptación

Alcance total del plan

Aplicación	Archivos	Líneas aprox.	Tests previos
apps/OpenSignServer (Parse Server, Jasmine)	83	~8.300	7 archivos de spec (43 specs)
apps/OpenSign (React + Vite, Vitest)	275	~60.600	14 archivos de test

Criterio acordado

- **Módulos críticos** (autenticación, borrado de cuenta, tenant y permisos, ciclo de vida de documentos, motor de firma PDF): **100 % de ramas**, junto con 100 % de líneas y funciones.
- **Resto** (correo, contactos, reportes, preferencias, UI no crítica): **80 %** de líneas y ramas.
- Ejecución por lotes, con revisión y reporte entre lotes, commit por lote y sin push.
- Los bugs encontrados **no se corrigen** en la rama de tests: se registran como defectos conocidos y se reportan.

3. Metodología y herramientas

Entorno de pruebas

- Los specs levantan un **Parse Server real** en `localhost:30001/test` contra MongoDB en `mongodb://localhost:27017/parse-test` (definido en `spec/utis/test-runner.js`).
- Se usa un contenedor Docker aislado, `fibesign-mongo-test` (puerto 27017), exclusivo para pruebas. El contenedor de desarrollo `mongo-container` (puerto 27018, base `OpenSignDB`) **no se usa**, para no poner en riesgo los datos de desarrollo: la suite ejecuta `deleteEverything()` al terminar.
- Todo I/O externo (SMTP/Mailhog, Gmail, S3) se simula con spies. No hay tráfico de red fuera de localhost.

Medición de cobertura

- `nyc` (script `coverage` existente) reporta **0/0** porque el servidor es ESM (`"type": "module"`) y `nyc` no lo instrumenta.
- Se agregó `c8` (cobertura nativa de V8) como devDependency y el script `test:coverage` .
- **Advertencia de V8:** reporta 100 % de ramas en archivos cuyas funciones nunca se ejecutaron. Por eso cada archivo se valida con las tres métricas a la vez: líneas, funciones y ramas.
- El prefijo `TESTING=true` de los scripts no funciona con `npm run` en `cmd` de Windows (le pasa igual al script `coverage` previo). Desde bash funciona.

Técnica de caja blanca

- Cada `if / else` , ternario, `try / catch` , `|| / ??` por defecto y encadenamiento opcional se ejercita en ambas direcciones.
- Las ramas difíciles de alcanzar (fallas de base de datos, del transporte de correo o de HTTP externo) se fuerzan con spies **filtrados por clase** (`interceptQuery` , `stubFirstFor` , `rejectFirstFor` , `rejectFindFor` , `stubGetFor`), y cada spec verifica que el spy fue efectivamente alcanzado.
- El trabajo en segundo plano (por ejemplo la reasignación masiva de `UpdateExistUserAsAdmin`) se espera consultando el **estado de la base de datos**, no mensajes de log.

Registro de defectos: `knownDefect`

Helper en `spec/utils/known-defect.js` : `knownDefect(id, descripción, cuerpo)` .

- Ejecuta el cuerpo del test con un callback `check(condición, mensaje)` .
- Si alguna verificación falla, el defecto sigue presente → la spec queda **pendiente** con `KNOWN DEFECT <id>` .
- Si todas pasan, el defecto fue corregido → la spec **falla** con "Known defect appears fixed; remove the marker". Así nunca queda un marcador desactualizado.
- Si el cuerpo no hace ninguna verificación, falla. Si lanza una excepción, se reporta como fallo real.

4. Línea base de cobertura

Medición con `c8` sobre `cloud/**` e `index.js` antes de comenzar (43 specs, 0 fallos):

Métrica	Servidor completo
Líneas / sentencias	17,9 %
Funciones	14,6 %
Ramas	58,8 % (inflado por la advertencia de V8 descrita arriba)

La mayoría de los archivos estaban entre 0 % y 10 % de líneas. Los únicos con cobertura relevante eran los tocados por el endurecimiento de seguridad previo (`AddAdmin` , `AuthLoginAsMail` , `usersignup` , `createUserAccount`).

5. Resultados del Lote 1

Cobertura por archivo (líneas / funciones / ramas)

Archivo (<code>cloud/parsefunction/</code>)	Antes	Después
<code>loginUser.js</code>	7,7 / 0 / —	100 / 100 / 100
<code>AuthLoginAsMail.js</code>	86 / 100 / 68	100 / 100 / 100
<code>usersignup.js</code>	83 / 100 / 38	100 / 100 / 100
<code>AddAdmin.js</code>	87 / 100 / 31	100 / 100 / 100
<code>addUser.js</code>	0,6 / 0 / —	100 / 100 / 100
<code>UpdateExistUserAsAdmin.js</code>	2,2 / 0 / —	100 / 100 / 100
<code>CheckAdminExist.js</code>	10 / 0 / —	100 / 100 / 100
<code>shared/createUserAccount.js</code>	92 / 100 / 83	100 / 100 / 100
<code>resetPassword.js</code>	1,6 / 0 / —	100 / 100 / 100
<code>VerifyEmail.js</code>	2 / 0 / —	100 / 100 / 100
<code>SendMailOTPv1.js</code>	65 / 50 / 40	100 / 100 / 100
<code>getUserDetails.js</code>	66 / 100 / 30	100 / 100 / 100
<code>isextenduser.js</code>	20 / 0 / —	100 / 100 / 100
<code>getUserId.js</code>	5,6 / 0 / —	100 / 100 / 100

"—" indica ramas no medibles porque la función nunca se ejecutó.

Suite

- **273 specs, 0 fallos, 14 pendientes** (los defectos conocidos).
- 8 corridas consecutivas en verde después de la revisión de calidad.
- Archivos nuevos: `spec/parsefunction/*.spec.js` (15 specs; `addUser` dividido en `authorization` y `creation`), `spec/utils/auth-fixtures.js`, `spec/utils/known-defect.js`, sus specs, y `spec/utils/otp-sender-probe.mjs` (proceso hijo para cubrir las dos ramas de `SMTP_ENABLE` sin depender del `.env` local).

6. Fallas de seguridad encontradas (SEC)

Cada falla fue **confirmada empíricamente**: su spec falla contra el código actual. La gravedad fue asignada por un revisor de seguridad independiente.

ID	Gravedad	Archivo	Falla	Quién puede explotarla
SEC-05	CRÍTICA	<code>addUser.js</code> (catch de error 202)	Si el email ya existe, ejecuta <code>setPassword</code> con master key sobre ese <code>_User</code> , sin verificar que pertenezca al tenant del llamante.	Admin u OrgAdmin de cualquier tenant; en la práctica, un anónimo (ver cap. 7).
SEC-03	ALTA (por cadena)	<code>VerifyEmail.js</code>	Ignora <code>ExpiresAt</code> y <code>FailedAttempts</code> ; <code>/functions/verifyemail</code> no está en la lista de rate limit estricto.	Cualquier usuario autenticado (el registro es abierto).
SEC-06	MEDIA (ALTA si las organizaciones son clientes distintos)	<code>resetPassword.js</code>	Solo filtra por tenant; un OrgAdmin puede resetear la contraseña de cualquier no-Admin del tenant, incluso de otras organizaciones. <code>addUser.js</code> sí filtra por organización.	OrgAdmin.
SEC-01	MEDIA	<code>AuthLoginAsMail.js</code>	Tras un login exitoso, el OTP no se invalida ni se reinicia <code>FailedAttempts</code> : se puede reutilizar durante los 10 minutos de vigencia.	Quien conozca el OTP (buzón, oráculo de SEC-03).
SEC-02	BAJA a MEDIA	<code>VerifyEmail.js</code>	Busca el OTP por el parámetro <code>email</code> y nunca lo compara con <code>request.user</code> : cualquier OTP emitido marca como verificada la cuenta del llamante.	Cualquier usuario autenticado.
SEC-07	BAJA a MEDIA	<code>usersignup.js</code>	El rol se valida después de <code>createUserAccount</code> ; si falta, lanza <code>TypeError</code> y deja un <code>_User</code> huérfano con sesión activa.	Anónimo (rate limit 30 cada 5 min por IP).
SEC-04	BAJA	<code>addUser.js</code>	La respuesta incluye la contraseña en texto plano del usuario creado, dentro de <code>UserId</code> .	Admin u OrgAdmin (la contraseña la eligió él, pero queda expuesta en la respuesta y en logs).

Nota sobre SEC-04

El revisor de seguridad lo descartó al leer el código, pero la verificación empírica lo confirma. La respuesta de `adduser` contiene:

```
"UserId":{"name":"New Member","username":"member-...@example.com",  
  "email":"member-...@example.com","password":"Member-Password!", ...}
```

El objeto `_User` completo, contraseña incluida, queda incrustado al serializar `extUserRes`.

7. Cadena de ataque: toma de cuenta por email

Cadena A — SEC-05 (crítica)

1. Un anónimo llama a `usersignup` con `role: "contracts_Admin"` y un tenant nuevo: queda como Admin de su propio tenant (el `role` no tiene lista blanca).
2. Crea una organización y un equipo en ese tenant.
3. Llama a `adduser` con el **email de la víctima** y una contraseña elegida por él.
4. La creación falla con error 202 (el email ya existe) y el `catch` **sobrescribe la contraseña de la víctima** con master key.
5. El atacante inicia sesión como la víctima. Además, la víctima queda con una segunda fila `contracts_Users` en el tenant del atacante, lo que puede mezclar datos entre tenants en código que use `.first()` sobre `UserId`.

Cadena B — SEC-03 + SEC-01 (alta a crítica)

1. Un anónimo llama a `SendOTPMailV1(emailVíctima)`: se emite un código de 4 dígitos (1000–9999) y se reinicia `FailedAttempts` a 0.
2. Un usuario autenticado cualquiera llama a `verifyemail(emailVíctima, otp)` en bucle sobre los ~9.000 códigos. Nada limita los intentos.
3. La respuesta distingue acierto ("Email is verified." o "already verified") de fallo ("OTP is invalid"): funciona como **oráculo**.
4. Dentro de los 10 minutos de vigencia, `AuthLoginAsMail(emailVíctima, otp)` devuelve el `sessionToken` de la víctima a través de `/loginAs`.

`emailVerified` no se exige en ningún otro lugar (`verifyUserEmails: false`), así que el efecto de SEC-02 por sí solo tiene poco impacto; el riesgo real es la cadena.

8. Hallazgos adicionales de seguridad

Detectados por el revisor de seguridad en los mismos archivos; todavía no tienen spec de defecto conocido.

Gravedad	Archivo	Hallazgo
MEDIA	<code>usersignup.js</code>	<code>extClass</code> se deriva del <code>role</code> que envía el cliente (<code>role.split('_')[0]</code>): un anónimo puede escribir filas con master key en clases <code><x>_Users</code> arbitrarias (contaminación de esquema) y elegir su <code>UserRole</code> libremente.

MEDIA	<code>SendMailOTPv1.js</code>	Es anónimo: permite bombardear de correos cualquier dirección e invalidar el OTP pendiente de una víctima. El parámetro <code>docId</code> descuenta cuota de correo del dueño del documento. Guarda un <code>TenantId</code> controlado por el cliente. Devuelve el objeto <code>err</code> crudo. Si el envío falla, igual responde "Otp send".
MEDIA	<code>AuthLoginAsMail.js</code> / <code>SendMailOTPv1.js</code>	El bloqueo de 5 intentos se evade pidiendo un OTP nuevo (reinicia <code>FailedAttempts</code>). El espacio de 4 dígitos es pequeño. El rate limit es por IP y <code>trust proxy</code> está deshabilitado si no se define <code>TRUST_PROXY_HOPS</code> : detrás de un proxy, todos los clientes comparten una IP.
BAJA	<code>AuthLoginAsMail.js</code>	Envía la master key en cabeceras a <code>cloudServerUrl + /loginAs</code> : verificar que esa URL sea interna o HTTPS. El OTP se guarda en texto plano.
BAJA	<code>resetPassword.js</code>	No filtra <code>IsDisabled</code> en el llamante (un admin deshabilitado aún puede resetear). <code>.first()</code> puede elegir el tenant equivocado si hay varias filas. Sin política de contraseñas.
BAJA	<code>addUser.js</code>	Sin política ni verificación de fortaleza de contraseña. No asigna <code>normalizedEmail</code> , a diferencia de <code>createUserAccount</code> .
BAJA	<code>usersignup.js</code>	<code>console.log('Err ', err)</code> puede registrar contenido de la solicitud.

9. Defectos de manejo de errores (DEF)

Registrados con `knownDefect` ; las specs verifican el contrato correcto y quedan pendientes mientras el defecto exista.

ID	Archivo	Defecto
DEF-01	<code>AuthLoginAsMail.js</code>	La rama de "guardado vacío" llama a <code>reject()</code> , que no está declarado en ese scope. Lanza <code>ReferenceError</code> , que el <code>catch</code> externo convierte en <code>'Result not found'</code> .
DEF-02	<code>SendMailOTPv1.js</code>	Las fallas internas (búsqueda del OTP, parámetros faltantes) resuelven con el objeto de error como valor en lugar de rechazar. Parse lo serializa como <code>{}</code> .
DEF-03	<code>getUserId.js</code>	Mismo patrón: sin coincidencia resuelve con un <code>TypeError</code> ; una falla de consulta resuelve con la falla.
DEF-04	<code>addUser.js</code>	Se filtra un mensaje crudo de <code>TypeError</code> cuando el username ya existe con otro email.
DEF-05	<code>UpdateExistUserAsAdmin.js</code>	Se filtra un mensaje crudo de <code>TypeError</code> cuando la fila <code>contracts_Users</code> encontrada no tiene tenant.

Otro comportamiento a revisar

- `UpdateExistUserAsAdmin` lanza `updateUserExceptAdmin` **sin** `await` y reasigna **todas** las filas de `contracts_Users` al tenant y organización del admin promovido. Es comportamiento intencional del flujo de instalación, pero corre en segundo plano sin que el llamante sepa si terminó.
- `isextenduser` compara emails distinguiendo mayúsculas y minúsculas. Quedó registrado como spec de caracterización; confirmar si es deliberado.

10. Código muerto e inalcanzable

Ramas que solo se alcanzan con mocks. Están agrupadas en bloques `defensive branches` (`unreachable` via `public API`) y son candidatas a eliminarse:

Archivo	Rama
<code>usersignup.js</code>	"User already exist": <code>createUserAccount</code> ya falla si el usuario existe.
<code>AddAdmin.js</code>	"User already exist"; omisión de <code>addTeamAndOrg</code> si ya hay organización; omisión si la fila no se puede leer (se crea inmediatamente antes).
<code>addUser.js</code>	<code>if (user)</code> falso: <code>Parse.User.save</code> siempre resuelve con el usuario.

<code>UpdateExistUserAsAdmin.js</code>	Guardado de <code>contracts_Users</code> que devuelve vacío.
<code>VerifyEmail.js</code>	Guardado del usuario que devuelve vacío.
<code>AuthLoginAsMail.js</code>	Guardado del usuario que devuelve vacío (ver DEF-01).
<code>getUserId.js</code>	Archivo completo: no está registrado en <code>cloud/main.js</code> . Una spec verifica que <code>Parse.Cloud.run('getUserId')</code> falla.

11. Revisión de calidad de las pruebas

Un revisor independiente evaluó si las pruebas detectarían regresiones reales, no solo si suben la cobertura.

Hallazgos y corrección

Nivel	Hallazgo	Corrección aplicada
ALTO	Carrera en <code>UpdateExistUserAsAdmin.spec.js</code> : la reasignación en segundo plano (lotes de 100) no se esperaba; causa probable de un fallo intermitente (1 de 7 corridas).	Seguimiento de <code>saveAll</code> (lotes iniciados y terminados); espera por estado de la base; timeouts explícitos.
MEDIO	Spies sobre <code>Parse.Query.prototype</code> sin filtro de clase: interceptaban cualquier consulta, incluidas las de triggers.	Helpers filtrados por clase con verificación de <code>hits()</code> .
MEDIO	Specs que fijaban defectos como comportamiento esperado (por ejemplo el <code>ReferenceError</code> de DEF-01).	Convertidas en defectos conocidos DEF-01 a DEF-05.
MEDIO	Aserciones débiles (<code>not.toBeNull()</code>).	Verifican código, mensaje y ausencia de efectos secundarios.
MEDIO	Timeouts de espera iguales al timeout de Jasmine; proceso hijo sin timeout.	<code>waitUntil</code> a 4000 ms por defecto; timeouts explícitos de 30 s en el proceso hijo.
MEDIO	Limpieza de <code>contracts_Users</code> cortada en 1000 filas.	Borrado en bucle hasta vaciar; verificación por ids concretos.
MEDIO	<code>pending()</code> antes del Arrange: el cuerpo nunca se ejecutaba y citaba números de línea desfasados.	Helper <code>knownDefect</code> (ver cap. 3), sin números de línea.
BAJO	URLs y claves de test fijas en el código; <code>console.error</code> silenciado de forma global.	Uso de <code>process.env</code> ; solo se captura <code>console.error</code> donde se verifica.

Restricción conocida

Las specs de `CheckAdminExist` y `UpdateExistUserAsAdmin` borran todas las filas `contracts_Users` antes de cada test y requieren `random: false` en `spec/support/jasmine.json` (configuración actual).

12. Recomendaciones y orden de corrección

Abrir una rama `fix/auth-security` desde `staging`, separada de la rama de tests. Al corregir cada falla, su spec `knownDefect` fallará con "appears fixed": en ese momento se quita el marcador y la spec pasa a ser una prueba de regresión normal.

Prioridad	ID	Corrección sugerida
1	SEC-05	En el error 202, no tocar la contraseña . Rechazar con "el usuario ya existe", o vincular la cuenta solo si ya pertenece al tenant del llamante.
2	SEC-03	Aplicar la misma expiración y bloqueo que <code>AuthLoginAsMail</code> (un helper compartido); agregar <code>/functions/verifyemail</code> al rate limit estricto; consumir el OTP al acertar.
3	SEC-01	Eliminar o anular la fila <code>defaultdata_Otp</code> tras el login exitoso; guardar un hash del OTP.
4	SEC-06	Para llamantes OrgAdmin, agregar <code>equalTo('OrganizationId', ...)</code> sobre el usuario objetivo.
5	SEC-07 y <code>role</code> libre	Validar <code>userDetails</code> con un esquema antes de <code>createUserAccount</code> ; lista blanca de <code>role</code> .
6	SEC-02	Exigir que <code>email</code> coincida (normalizado) con <code>request.user.get('email')</code> .
7	SEC-04	No devolver el <code>_User</code> incrustado; responder solo con los campos necesarios.
8	<code>SendMail0TPv1</code>	No reiniciar <code>FailedAttempts</code> al reemitir; limitar por destinatario; no devolver <code>err</code> crudo; OTP de 6 dígitos.
9	DEF-01 a DEF-05	Rechazar con <code>Parse.Error</code> y mensajes controlados en lugar de resolver con errores o filtrar <code>TypeError</code> .
10	Código muerto	Eliminar <code>getUserId.js</code> y las ramas inalcanzables del capítulo 10.

Otros pendientes de herramientas:

- Hacer multiplataforma los scripts `coverage` y `test:coverage` (por ejemplo con `cross-env`).
- Agregar `eslint.config.js` : `npx eslint` no puede correr en el servidor.
- `.env` local con `SMTP_ENABLE=true` puede diferir de CI; las specs ya cubren ambos valores.

13. Plan de los lotes siguientes

Servidor (`apps/OpenSignServer`)

Lote	Módulos	Criterio	Estado
1	Autenticación y alta de usuarios (14 archivos)	100 % ramas	Completado (2f9cc4f4)
2	Borrado de cuenta (<code>customRoute/deleteAccount/*</code>), tenant y permisos (<code>getTenant</code> , <code>updateTenant</code> , <code>getUserListByOrg</code> , <code>getTeams</code> , <code>*AfterFind</code>)	100 % ramas	Pendiente
3	Ciclo de vida de documentos: <code>createDocumentFromApp</code> , <code>createBatchDocs</code> , <code>getDocument</code> , <code>getSignedUrl</code> , <code>declinedocument</code> , <code>ForwardDoc</code> , <code>recreateDocument</code> , <code>saveAsTemplate</code> , <code>createDuplicate</code> , <code>linkContactToDoc</code> , triggers <code>Document*</code> y <code>Template*</code>	100 % ramas	Pendiente
4	Motor de firma: <code>pdf/PDF.js</code> , <code>GenerateCertificate</code> , <code>Placeholder</code> , <code>PDFArrayCustom</code> , <code>generateCertificatebydocId</code> , <code>decryptpdf</code> , <code>docxtopdf</code> , firmas	100 % ramas	Pendiente
5	Correo (<code>sendMail*</code> , <code>sendSystemMail</code>) y contactos	80 %	Pendiente
6	Reportes, preferencias, tours, archivos, logo, <code>main.js</code> , <code>index.js</code>	80 %	Pendiente

Frontend (`apps/OpenSign`)

Mismo esquema por lotes después del servidor. Críticos: utilidades, Redux y páginas de autenticación y firma. Al 80 %: `emailbuilder` y componentes visuales del PDF.

Reglas de cada lote

1. Un subagente escribe las pruebas de caja blanca con I/O externo simulado.
2. Revisión independiente de calidad de las pruebas (y de seguridad cuando aparecen fallas).
3. Verificación propia: suite completa en varias corridas y `c8` por archivo.
4. Commit por lote sin push, y reporte antes de continuar.